



香港科技大學(廣州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)



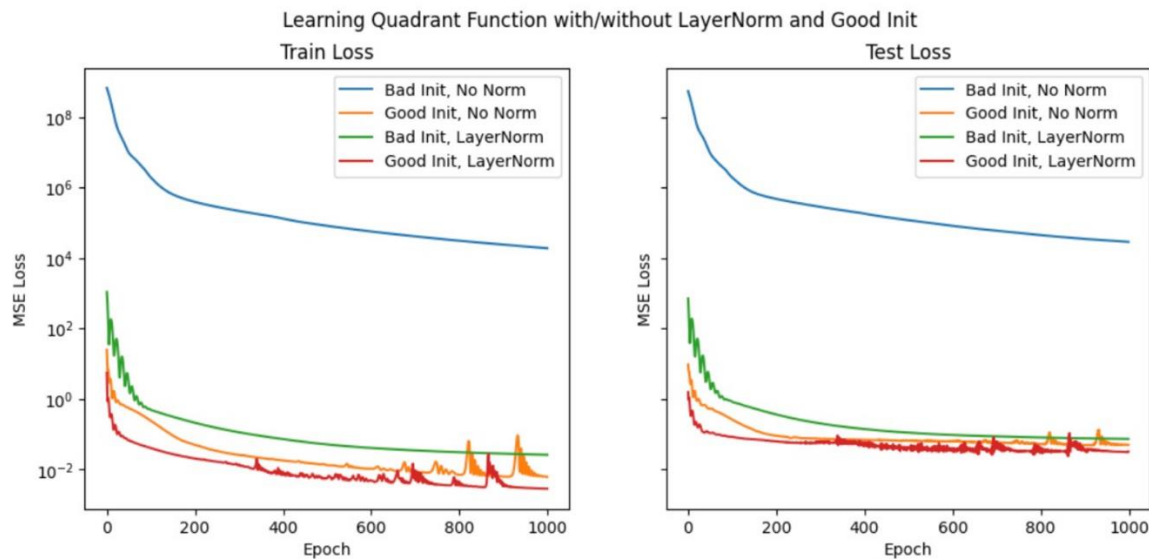
香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

AIAA 2205: Introduction to AI

Enyan Dai

The Hong Kong University of Science and Technology (Guangzhou)

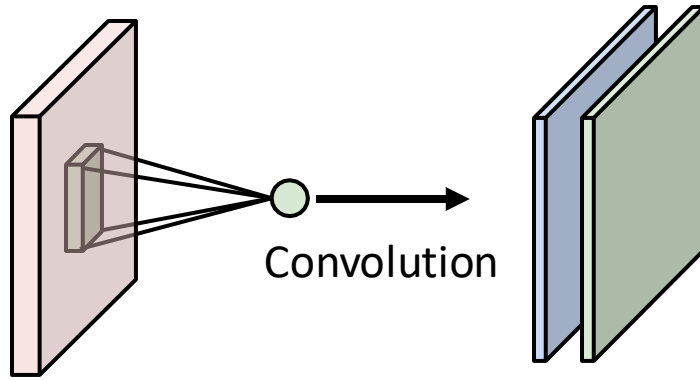
Can normalization resolve poor initialization?



Toy setting, 2d input and
2-layer NN with ReLU

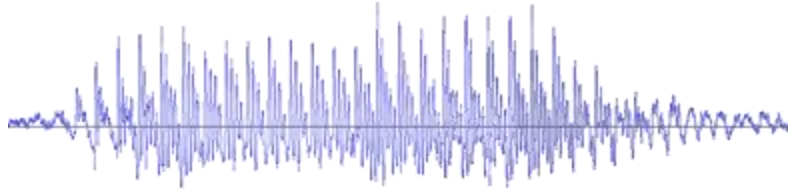
Image and Sequential Data

Activation maps



CNN preserves the spatial information

Signal:



Text:

How are you? I am fine. Thank you, and you?

But how to preserve the temporal information?

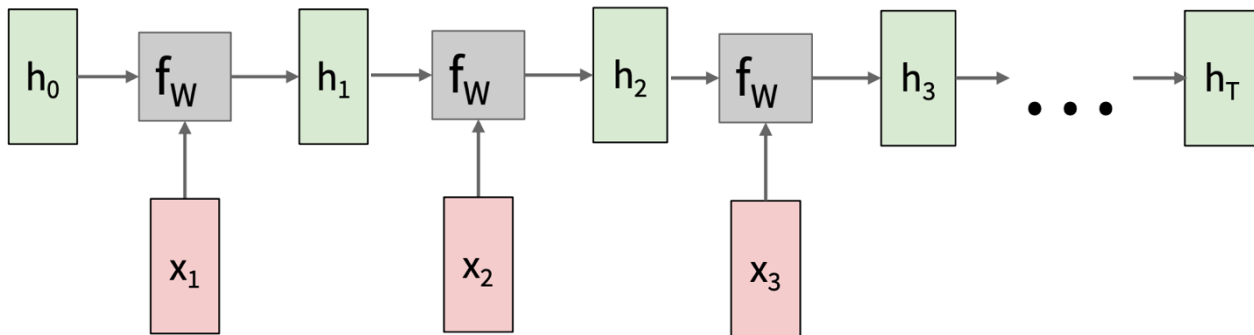
Recurrent Neural Network

Recurrent Neural Network (RNN) update the representation by combining:
previous step representation h_{t-1} and **current step input x_t**

New state $h_t = f_W(h_{t-1}, x_t)$

Learnable function

Initialized
hiddent state

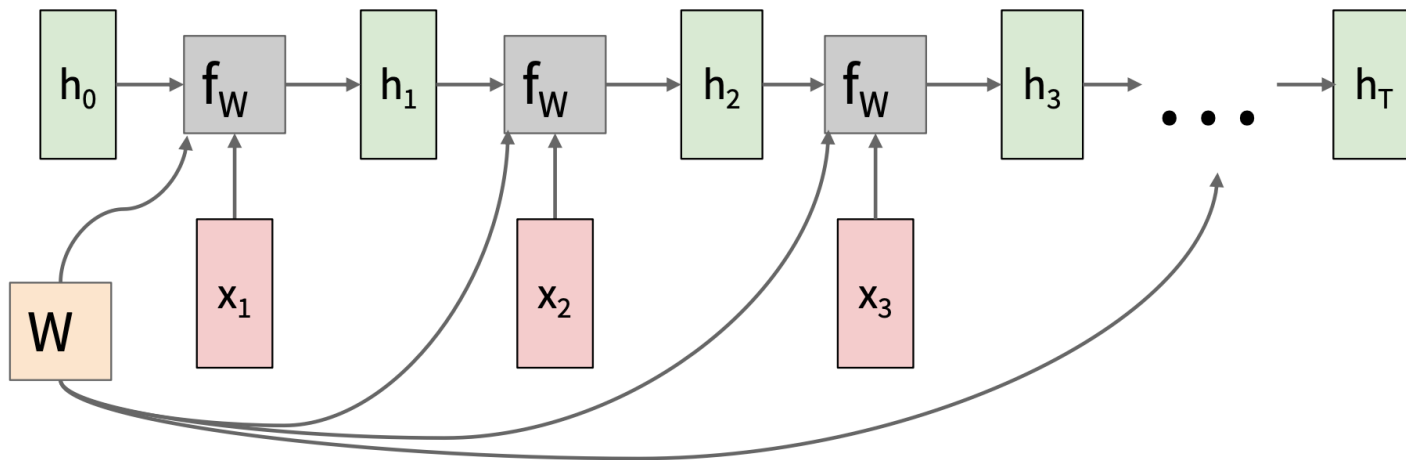


Data at step 1

Recurrent Neural Network

The **same parameters** are used at every time step

$$h_t = f_W(h_{t-1}, x_t)$$

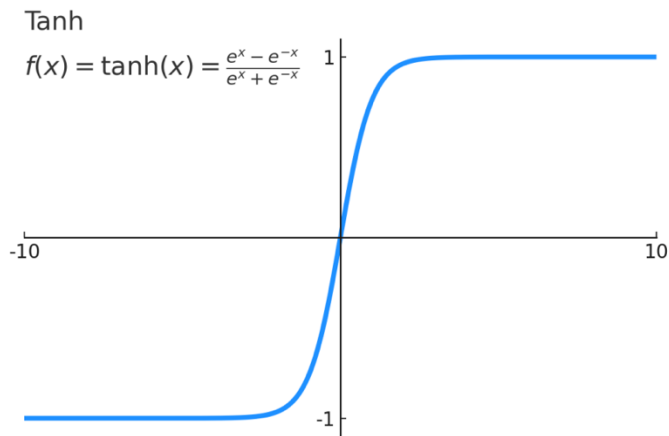
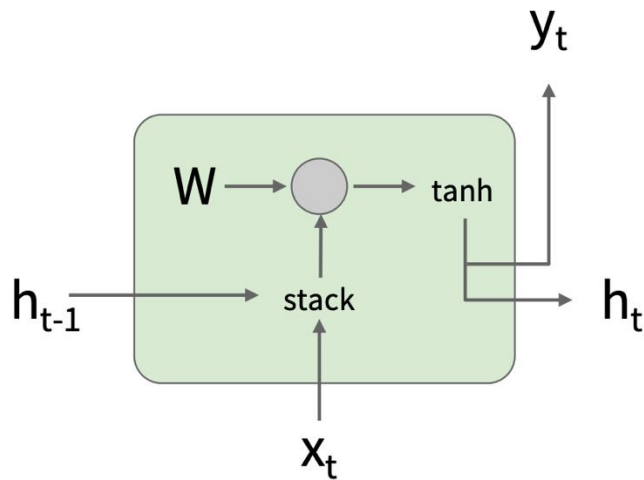


Let's Look into Some Details

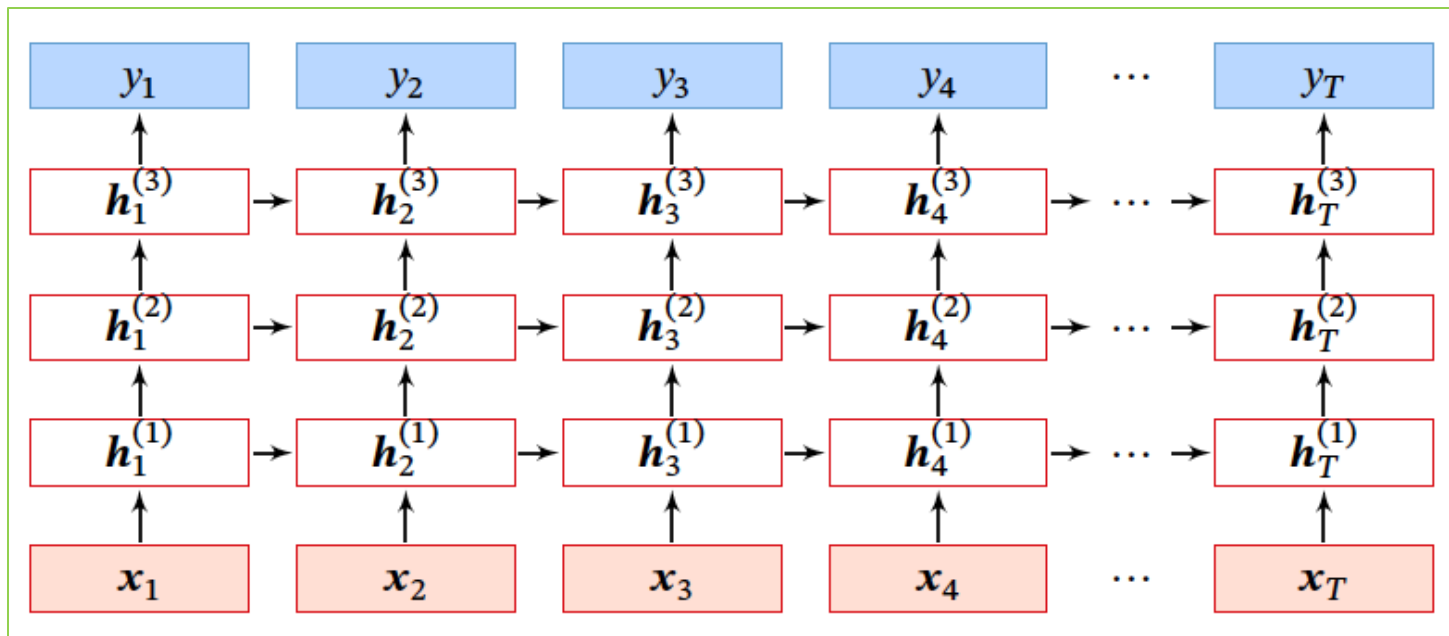
How to build the $h_t = f_W(h_{t-1}, x_t)$?

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$$

$$y_t = W_{hy}h_t$$

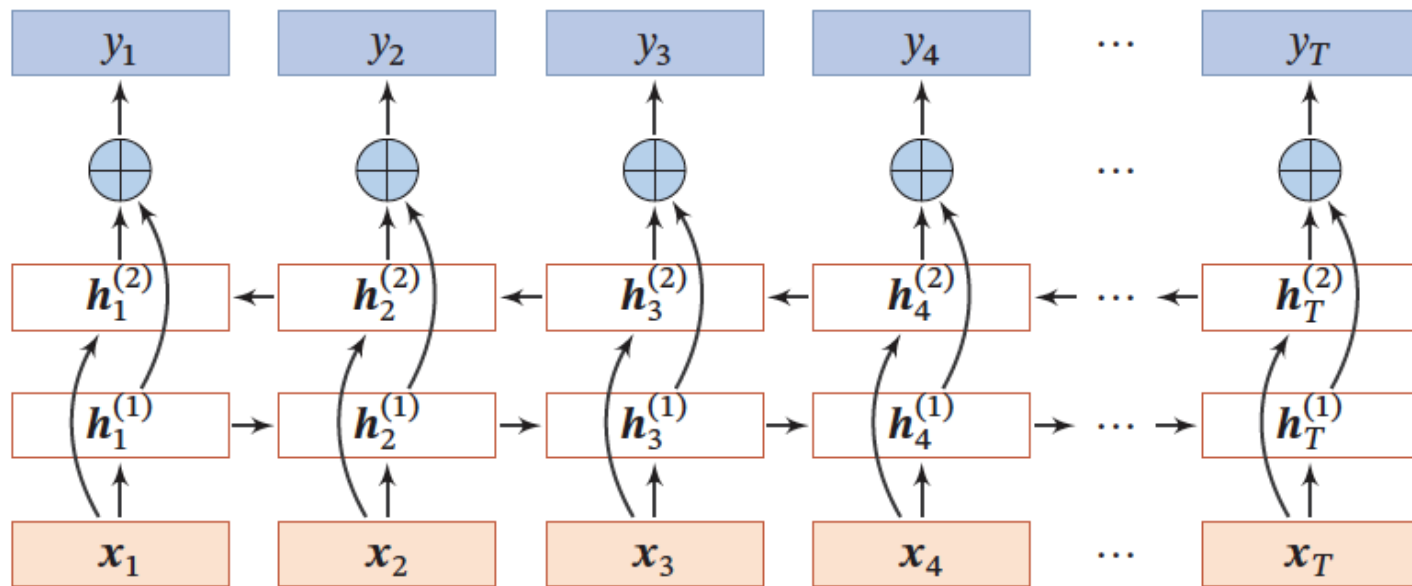


Deep stacked RNN



$$h_t^{(l)} = f(U^{(l)}h_{t-1}^{(l)} + W^{(l)}h_t^{(l-1)} + b^{(l)}),$$

Bidirectional RNN



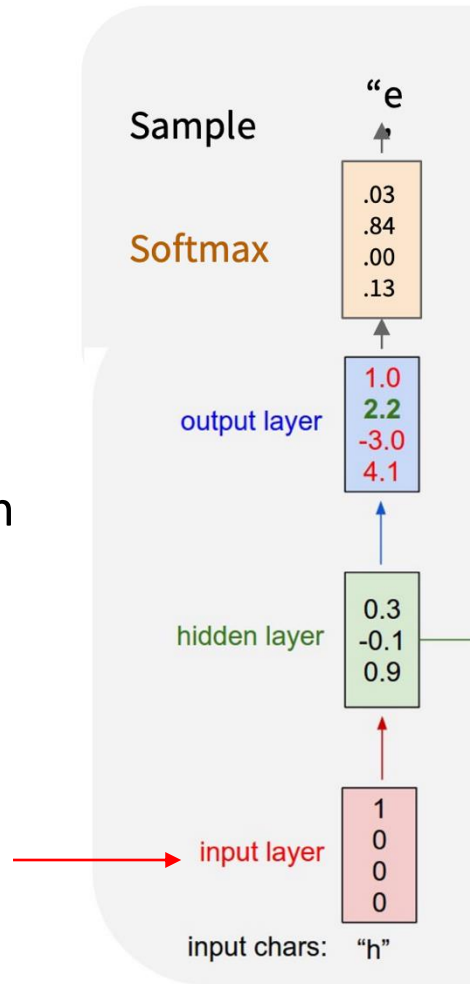
$$h_t^{(1)} = f(U^{(1)}h_{t-1}^{(1)} + W^{(1)}x_t + b^{(1)}),$$

$$h_t^{(2)} = f(U^{(2)}h_{t+1}^{(2)} + W^{(2)}x_t + b^{(2)}),$$

$$h_t = h_t^{(1)} \oplus h_t^{(2)},$$

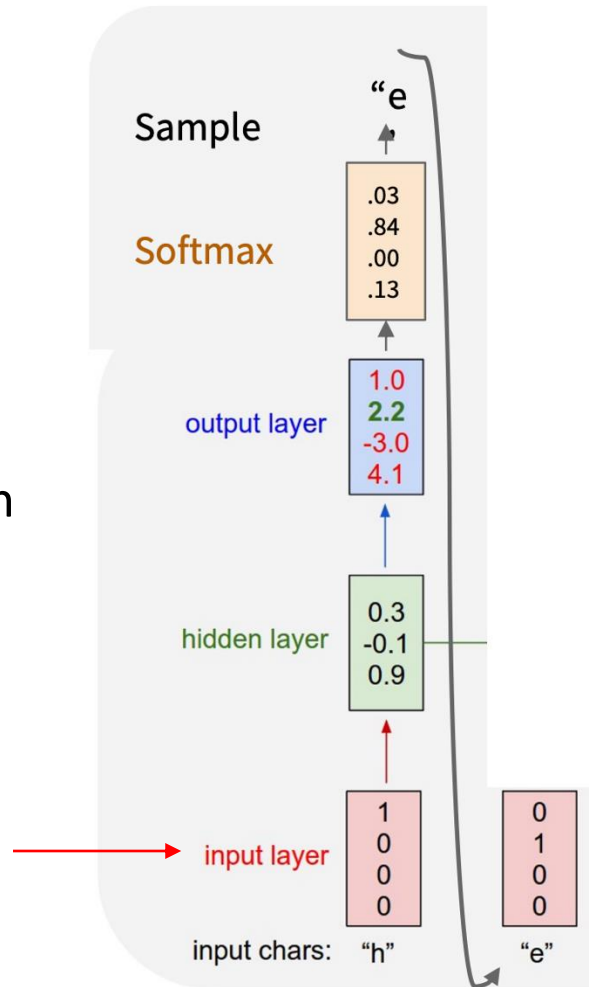
An example of Character-level Language Generation

One Hot
Vector



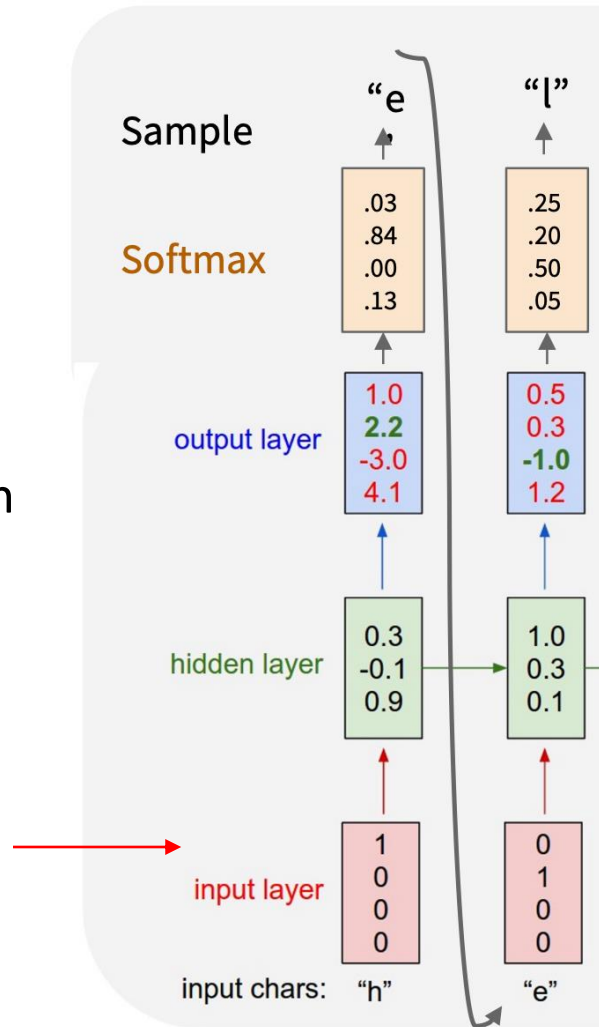
An example of Character-level Language Generation

One Hot
Vector



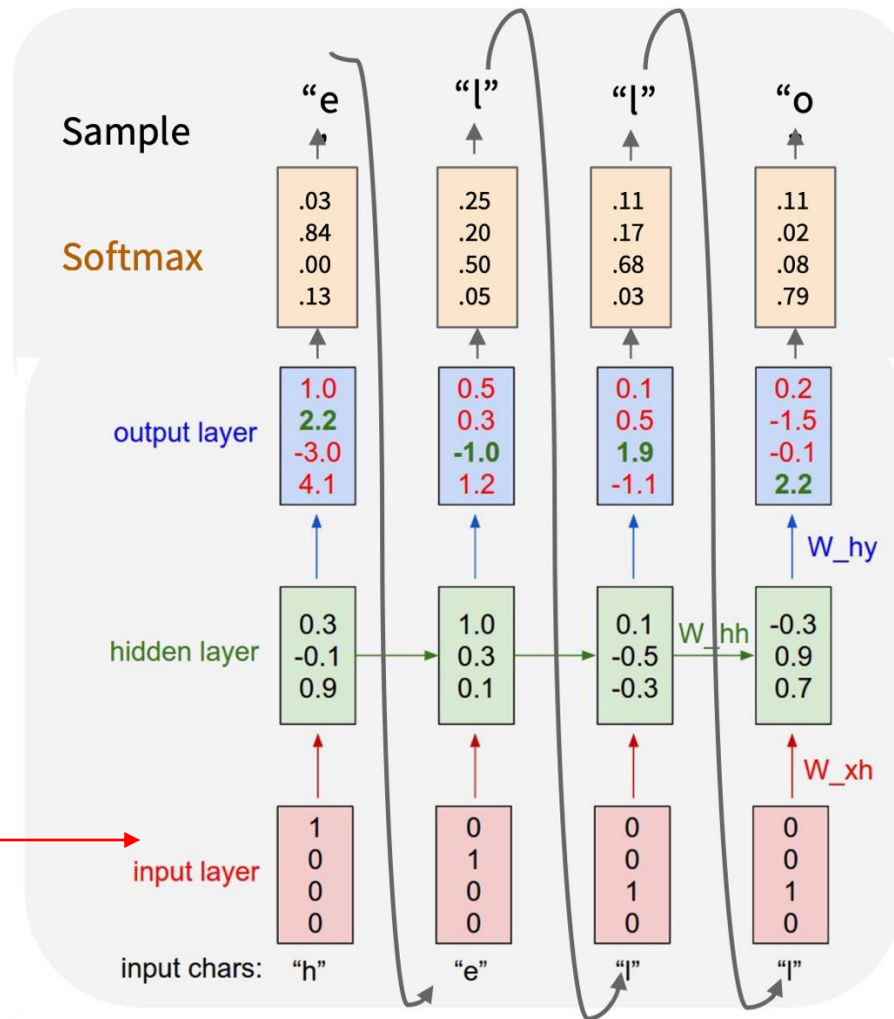
An example of Character-level Language Generation

One Hot
Vector



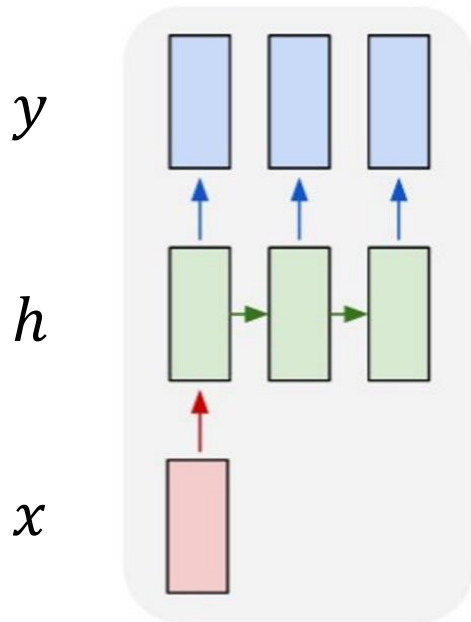
An example of Character-level Language Generation

One Hot Vector

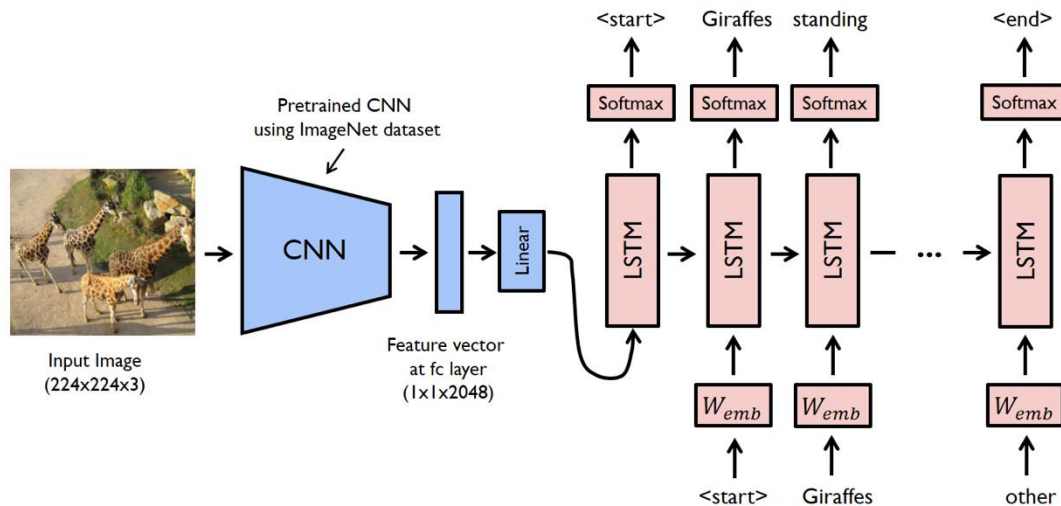


Recurrent Neural Networks

One to many

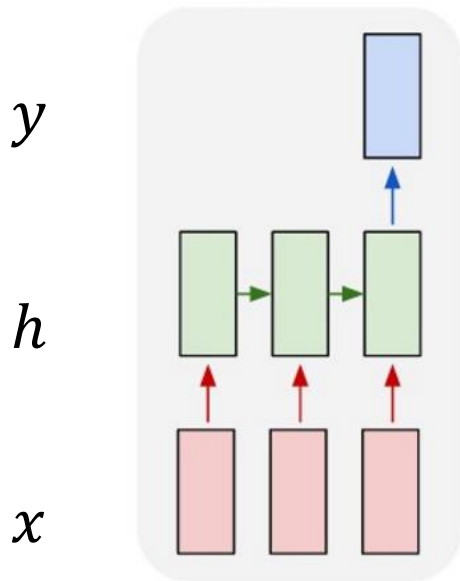


Example: Image captioning

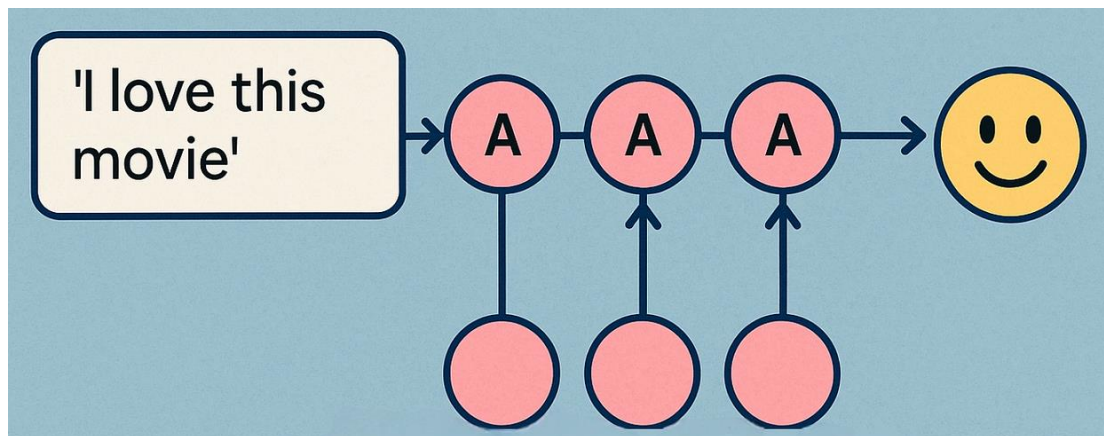


Recurrent Neural Networks

Many to one



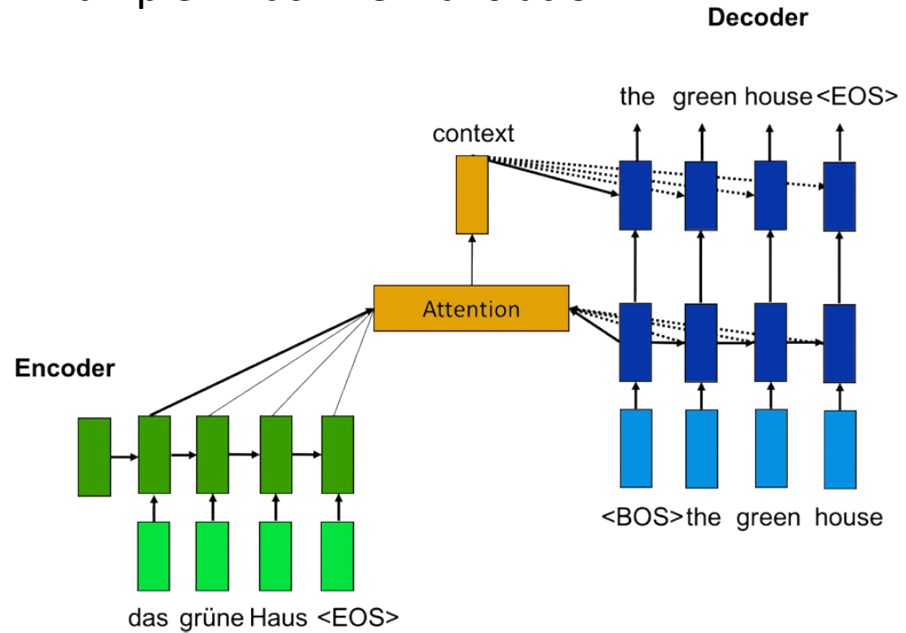
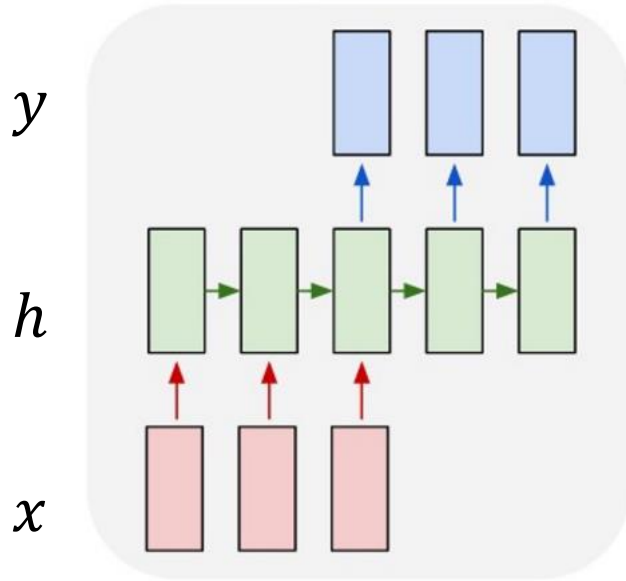
Example: Sentiment Analysis (情感分析)



Recurrent Neural Networks

Many to Many

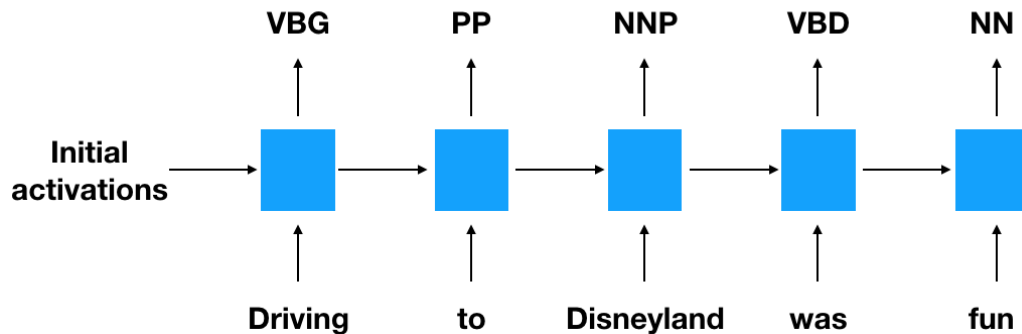
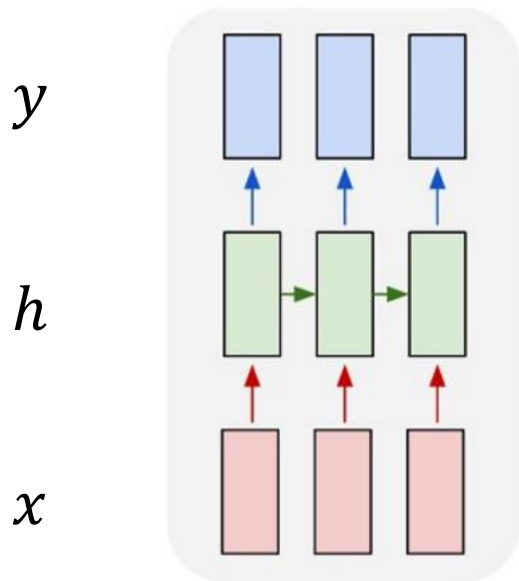
Example: Machine Translation

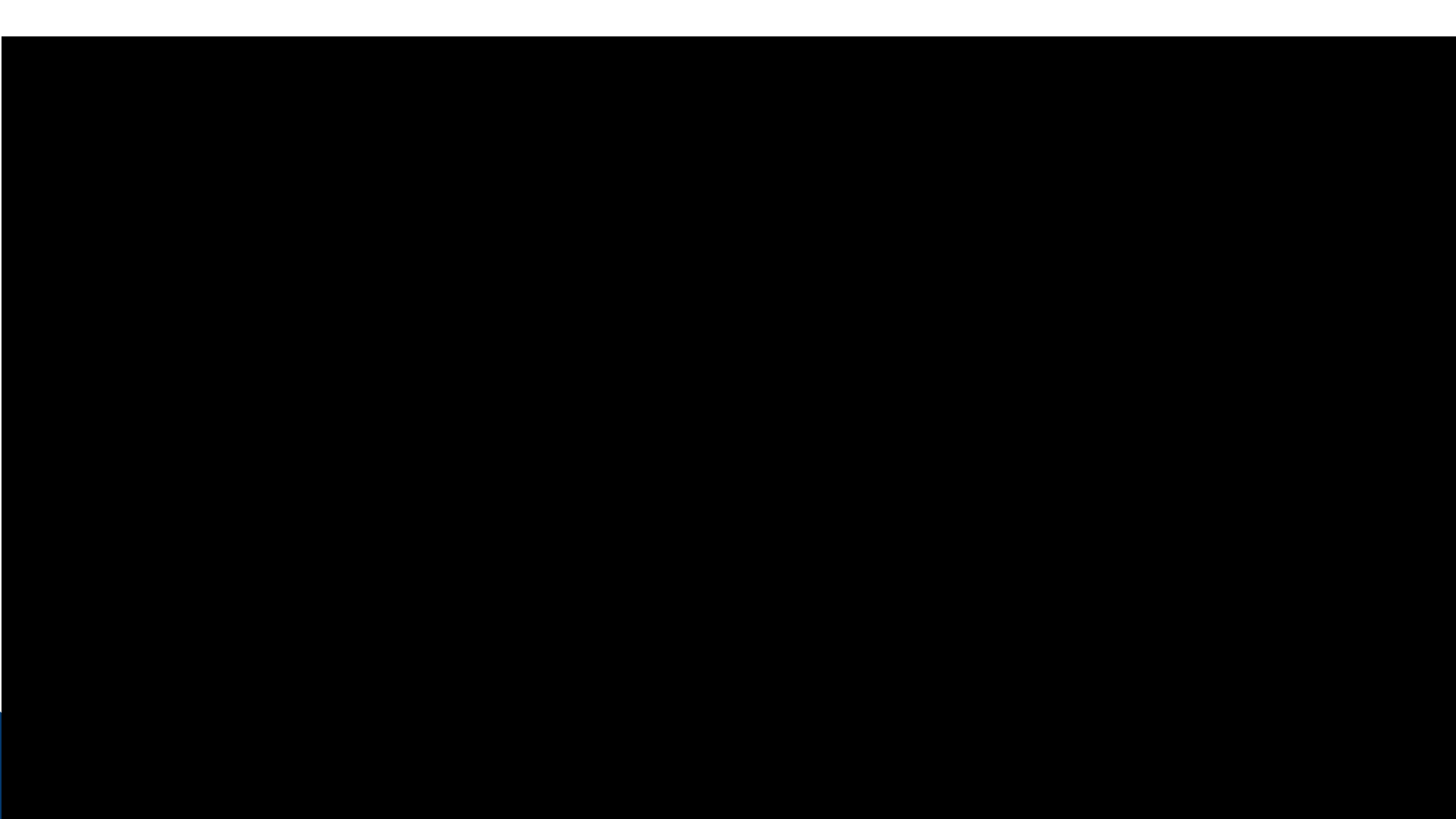


Recurrent Neural Networks

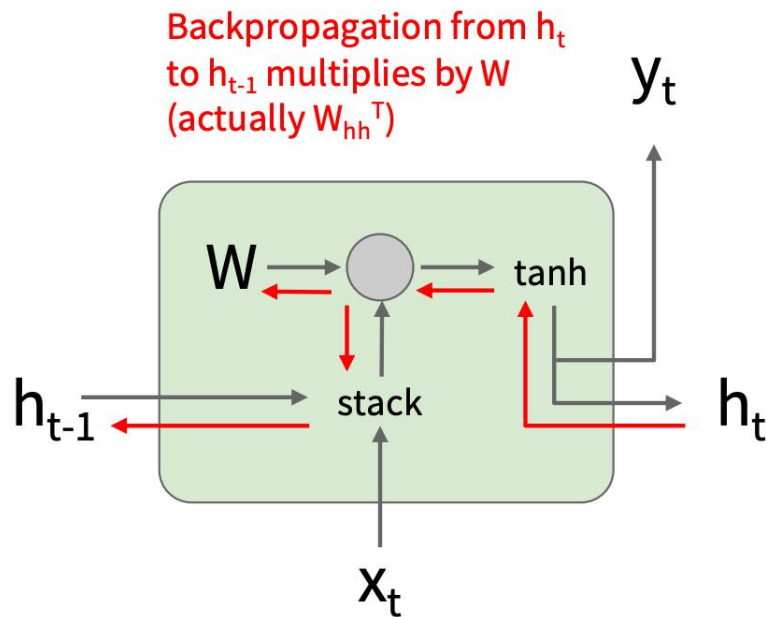
Many to Many

Example: POS Tagging





Gradient Flow in RNNs



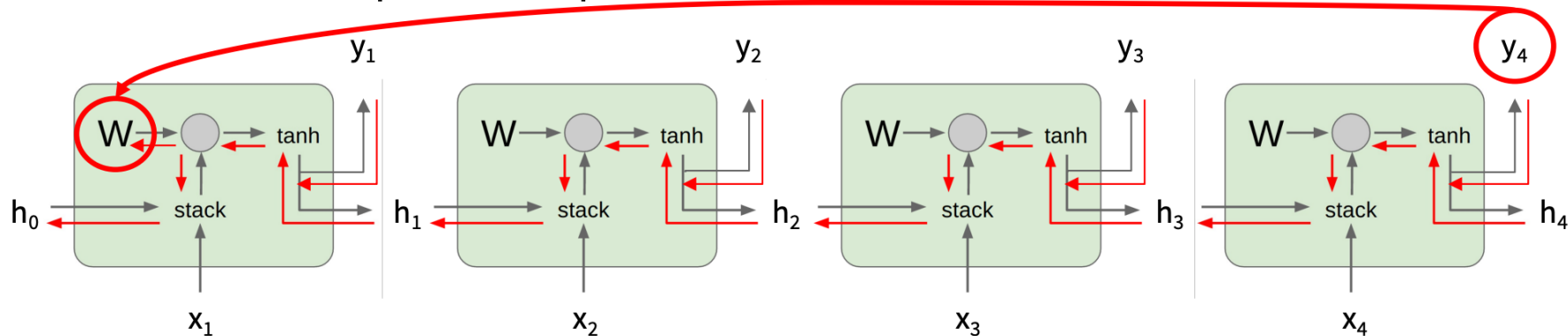
$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

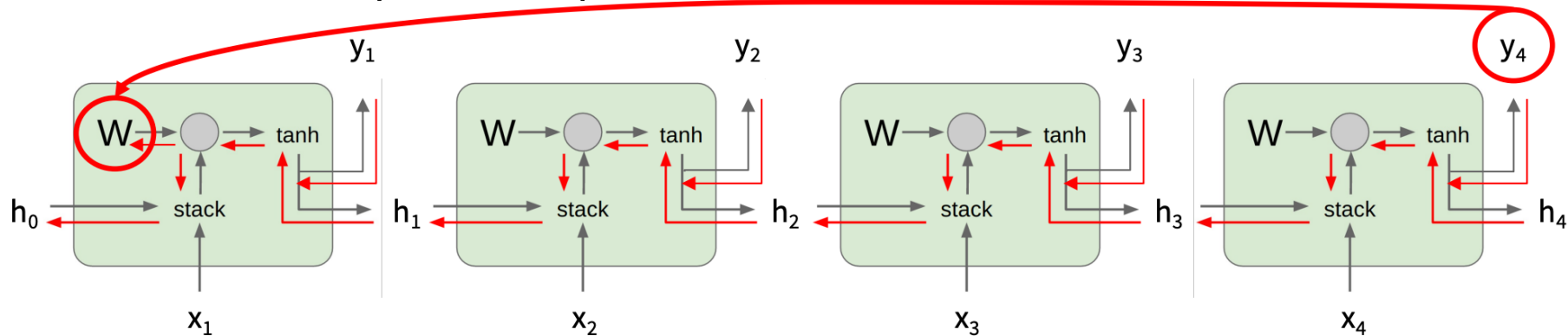
$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest value > 1 :
Exploding gradients

Gradient clipping

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

Largest value < 1 :
Vanishing gradients

Long Short Term Memory (LSTM)

RNN:

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

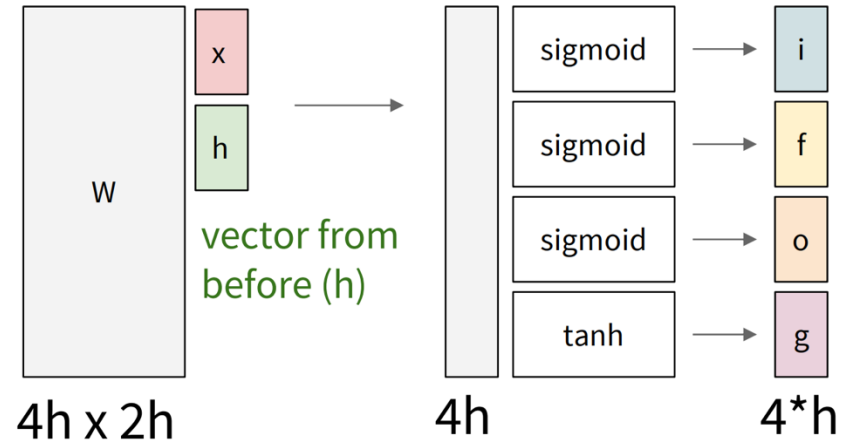
LSTM

Four gates $\rightarrow \begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$

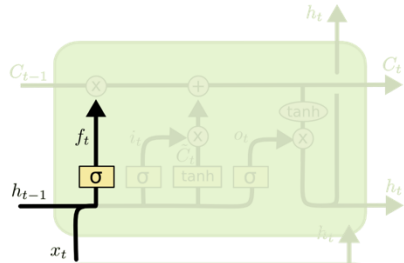
Cell state $\rightarrow c_t = f \odot c_{t-1} + i \odot g$

Hidden state $\rightarrow h_t = o \odot \tanh(c_t)$

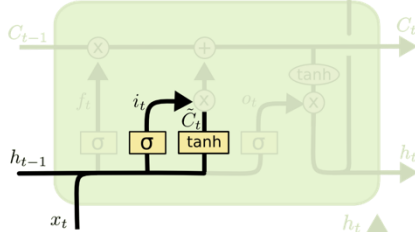
i: Input gate, whether to write to cell
 f: Forget gate, Whether to erase cell
 o: Output gate, How much to reveal cell
 g: Gate gate, How much to write to cell



LSTM

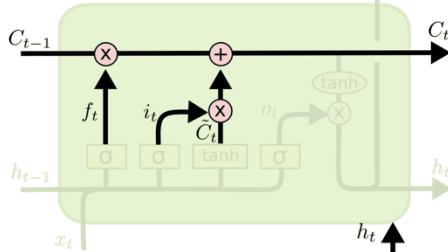


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

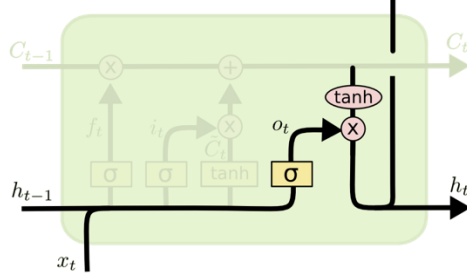


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

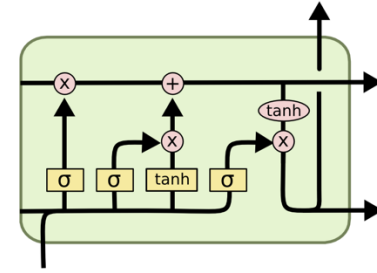


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



i_t decides what component is to be updated.

$\tilde{C}_t$ provides change contents

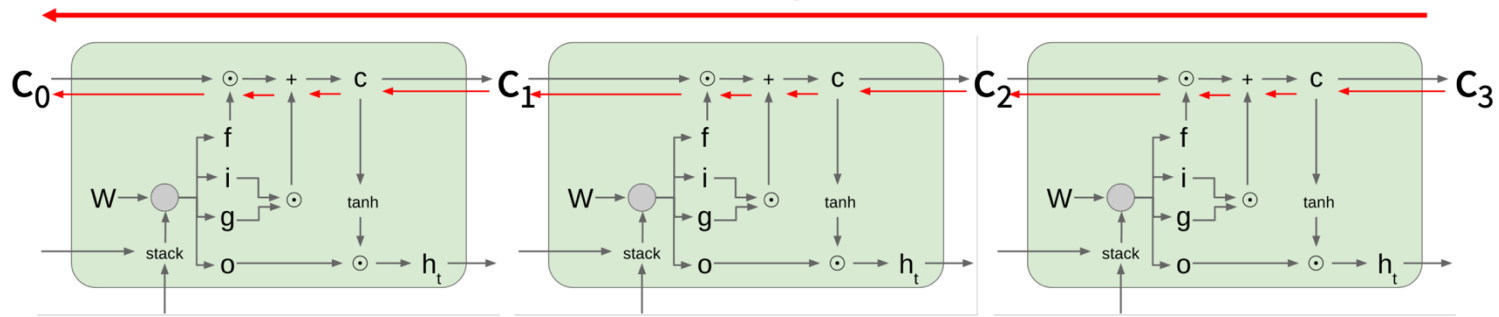
Updating the cell state

Decide what part of the cell state to output

Long Short Term Memory (LSTM)

The cell state **only has linear operations**, similar to the residual connection
Some people call “ResNet is LSTM that rotates 90 degree”

Uninterrupted gradient flow!



Similar to ResNet!

